Algoritmos e Estrutura de Dados II

Ordenação Interna - MergeSort e QuickSort

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Instituto de Ciências Exatas e Informática

Ordenação por Mistura - *Merge Sort*

- O *merge sort*, ou ordenação por mistura, é um exemplo de algoritmo de ordenação do tipo dividir-para-conquistar.
- Sua ideia básica consiste em Dividir (o problema em vários sub-problemas e resolver esses sub-problemas através da recursividade) e Conquistar (após todos os sub-problemas terem sido resolvidos ocorre a conquista que é a união das resoluções dos sub-problemas)
- Como o algoritmo do *Merge Sort* usa a recursividade em alguns problemas esta técnica não é muito eficiente devido ao alto consumo de memória e tempo de execução.

Ordenação por Mistura - *Merge Sort*

- Os três passos úteis dos algoritmos dividir-para-conquistar que se aplicam ao *merge sort* são:
  1. **Dividir**: Dividir os dados em subsequências pequenas;
  2. **Conquistar**: Classificar as duas metades recursivamente aplicando o **merge sort**;
  3. **Combinar**: Juntar as duas metades em um único conjunto já classificado.

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]
[	[3	1	5]	[4	2]	]

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]
[	[3	1	5]	[4	2]	]
[	[3	1]	[5]	[4	2]	]

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]
[	[3	1	5]	[4	2]	]
[	[3	1]	[5]	[4	2]	]
[	[3]	[1]	[5]	[4]	[2]	]

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]
[	[3	1	5]	[4	2]	]
[	[3	1]	[5]	[4	2]	]
[	[3]	[1]	[5]	[4]	[2]	]
[	[1	3]	[5]	[2	4]	]

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]
[	[3	1	5]	[4	2]	]
[	[3	1]	[5]	[4	2]	]
[	[3]	[1]	[5]	[4]	[2]	]
[	[1	3]	[5]	[2	4]	]
[	[1	3	5]	[2	4]	]

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]
[	[3	1	5]	[4	2]	]
[	[3	1]	[5]	[4	2]	]
[	[3]	[1]	[5]	[4]	[2]	]
[	[1	3]	[5]	[2	4]	]
[	[1	3	5]	[2	4]	]
[	**1**	**2**	**3**	**4**	**5**	]

Ordenação por Mistura - *Merge Sort*

```
1 public static void MergeSort(Item v[], int inicio, int fim) {
2   if (inicio < fim) {
3     int meio = (inicio + fim) / 2;
4     MergeSort(v, inicio, meio);
5     MergeSort(v, meio + 1, fim);
6     merge(v, inicio, meio, fim);
7   }
8 }
```

Ordenação por Mistura - *Merge Sort*

```
1  private static void merge(Item v[], int inicio, int meio, int fim) {
2    int n1, n2, i, j, k;
3    n1 = meio - inicio + 1;
4    n2 = fim - meio;
5
6    Item A1[] = new Item[n1 + 1];
7    Item A2[] = new Item[n2 + 1];
8
9    for (i = 0; i < n1; i++)
10     A1[i] = v[inicio + i];
11   for (j = 0; j < n2; j++)
12     A2[j] = v[meio + j + 1];
13
14   A1[i] = new Item (Integer.MAX_VALUE);
15   A2[j] = new Item (Integer.MAX_VALUE);
16
17   i = 0; j = 0;
18
19   for (k = inicio; k <= fim; k++) {
20     if (A1[i].chave() <= A2[j].chave())
21       v[k] = A1[i++];
22     else
23       v[k] = A2[j++];
24   }
25 }
```

Ordenação por Mistura - *Merge Sort*

- Número de Comparações
  - Melhor Caso: $n \times \log n$
  - Pior Caso: $n \times \log n$
- Número Movimentações
  - Melhor Caso: $n \times \log n$
  - Pior Caso: $n \times \log n$

Ordenação por Mistura - *Merge Sort*

- Vantagens:
  - *MergeSort* é $n \times \log n$
  - Indicado para aplicações que exigem restrição de tempo (executa sempre em um determinado tempo para um dado n)
  - Passível de ser transformado em estável
  - Fácil Implementação
- Desvantagens:
  - Utiliza funções recursivas
  - Gasto extra de memória, o algoritmo cria uma cópia do vetor para cada nível da chamada recursiva, totalizando um uso adicional de memória igual a n

Exercício

- Dada a sequência de números: [3 4 9 2 5 1 8] Ordene em ordem crescente utilizando o algoritmo *MegeSort*, apresentado a sequência dos números a cada passo.

Quick Sort

- Proposto por Hoare em 1960 e publicado em 1962.
- É o algoritmo de ordenação interna mais rápido que se conhece para uma ampla variedade de situações.
- Provavelmente é o mais utilizado.
- A ideia básica é dividir o problema de ordenar um conjunto com n itens em dois problemas menores.
- Os problemas menores são ordenados independentemente.
- Os resultados são combinados para produzir a solução final.

Quick Sort

- A parte mais delicada do método é o processo de partição.
- O vetor $A[Esq..Dir]$ é rearranjado por meio da escolha arbitrária de um **pivô** x.
- O vetor A é particionado em duas partes:
  - Parte esquerda: chaves $\leq$ x.
  - Parte direita: chaves $\geq$ x.

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	<u>3</u>	1	**5**	4	2	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	<u>3</u>	1	**5**	4	2	]
2	5	[	3	<u>1</u>	**5**	4	2	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	<u>3</u>	1	**5**	4	2	]
2	5	[	3	<u>1</u>	**5**	4	2	]
3	5	[	3	1	<u>**5**</u>	4	2	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	3	1	**5**	4	2	]
2	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	<u>3</u>	1	**5**	4	2	]
2	5	[	3	<u>1</u>	**5**	4	2	]
3	5	[	3	1	<u>**5**</u>	4	2	]
3	5	[	3	1	**5**	4	<u>2</u>	]
4	4	[	3	1	2	4	**5**	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	3	1	**5**	4	2	]
2	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
4	4	[	3	1	2	4	**5**	]
4	4	[	3	1	2	4	**5**	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	3	1	**5**	4	2	]
2	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
4	4	[	3	1	2	4	**5**	]
4	4	[	3	1	2	4	**5**	]
5	4	[	3	1	2	4	**5**	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	3	1	**5**	4	2	]
2	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
4	4	[	3	1	2	4	**5**	]
4	4	[	3	1	2	4	**5**	]
5	4	[	3	1	2	4	**5**	]
5	4	[	3	1	2	4	**5**	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	3	1	**5**	4	2	]
2	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
4	4	[	3	1	2	4	**5**	]
4	4	[	3	1	2	4	**5**	]
5	4	[	3	1	2	4	**5**	]
5	4	[	3	1	2	4	**5**	]
5	4	[	**3**	**1**	**2**	**4**	**5**	]

Quick Sort

Esq: 1
Dir: 4
Pivô: 1

i	j		1	2	3	4	5	
1	4	[	3	**1**	2	4	5	]

Quick Sort

Esq: 1
Dir: 4
Pivô: 1

i	j		1	2	3	4	5	
1	4	[	3	**1**	2	4	5	]
1	4	[	<u>3</u>	**1**	2	4	5	]

Quick Sort

Esq: 1
Dir: 4
Pivô: 1

i	j		1	2	3	4	5	
1	4	[	3	**1**	2	4	5	]
1	4	[	<u>3</u>	**1**	2	4	5	]
1	4	[	3	**1**	2	<u>4</u>	5	]

Quick Sort

Esq: 1
Dir: 4
Pivô: 1

i	j		1	2	3	4	5	
1	4	[	3	**1**	2	4	5	]
1	4	[	<u>3</u>	**1**	2	4	5	]
1	4	[	3	**1**	2	<u>4</u>	5	]
1	3	[	3	**1**	<u>2</u>	4	5	]

Quick Sort

Esq: 1
Dir: 4
Pivô: 1

i	j		1	2	3	4	5	
1	4	[	3	**1**	2	4	5	]
1	4	[	<u>3</u>	**1**	2	4	5	]
1	4	[	3	**1**	2	<u>4</u>	5	]
1	3	[	3	**1**	<u>2</u>	4	5	]
1	2	[	3	<u>**1**</u>	2	4	5	]

Quick Sort

Esq: 1
Dir: 4
Pivô: 1

i	j		1	2	3	4	5	
1	4	[	3	**1**	2	4	5	]
1	4	[	<u>3</u>	**1**	2	4	5	]
1	4	[	3	**1**	2	<u>4</u>	5	]
1	3	[	3	**1**	<u>2</u>	4	5	]
1	2	[	3	<u>**1**</u>	2	4	5	]
2	1	[	**1**	**3**	**2**	**4**	**5**	]

Quick Sort

Esq: 2
Dir: 4
Pivô: 2

i	j		1	2	3	4	5	
2	4	[	1	3	**2**	4	5	]

Quick Sort

Esq: 2
Dir: 4
Pivô: 2

i	j		1	2	3	4	5	
2	4	[	1	3	**2**	4	5	]
2	4	[	1	<u>3</u>	**2**	4	5	]

Quick Sort

Esq: 2
Dir: 4
Pivô: 2

i	j		1	2	3	4	5	
2	4	[	1	3	**2**	4	5	]
2	4	[	1	<u>3</u>	**2**	4	5	]
2	4	[	1	3	**2**	<u>4</u>	5	]

Quick Sort

Esq: 2
Dir: 4
Pivô: 2

i	j		1	2	3	4	5	
2	4	[	1	3	**2**	4	5	]
2	4	[	1	<u>3</u>	**2**	4	5	]
2	4	[	1	3	**2**	<u>4</u>	5	]
2	3	[	1	3	<u>**2**</u>	4	5	]

Quick Sort

Esq: 2
Dir: 4
Pivô: 2

i	j		1	2	3	4	5	
2	4	[	1	3	**2**	4	5	]
2	4	[	1	<u>3</u>	**2**	4	5	]
2	4	[	1	3	**2**	<u>4</u>	5	]
2	3	[	1	3	<u>**2**</u>	4	5	]
3	2	[	**1**	**2**	**3**	**4**	**5**	]

Quick Sort

Esq: 3
Dir: 4
Pivô: 3

i	j		1	2	3	4	5	
3	4	[	1	2	**3**	4	5	]

Quick Sort

Esq: 3
Dir: 4
Pivô: 3

i	j		1	2	3	4	5	
3	4	[	1	2	**3**	4	5	]
3	4	[	1	2	**<u>3</u>**	4	5	]

Quick Sort

Esq: 3
Dir: 4
Pivô: 3

i	j		1	2	3	4	5	
3	4	[	1	2	**3**	4	5	]
3	4	[	1	2	**<u>3</u>**	4	5	]
3	4	[	1	2	**3**	<u>4</u>	5	]

Quick Sort

Esq: 3
Dir: 4
Pivô: 3

i	j		1	2	3	4	5	
3	4	[	1	2	**3**	4	5	]
3	4	[	1	2	**<u>3</u>**	4	5	]
3	4	[	1	2	**3**	<u>4</u>	5	]
3	3	[	1	2	**<u>3</u>**	4	5	]

Quick Sort

Esq: 3
Dir: 4
Pivô: 3

i	j		1	2	3	4	5	
3	4	[	1	2	**3**	4	5	]
3	4	[	1	2	**<u>3</u>**	4	5	]
3	4	[	1	2	**3**	<u>4</u>	5	]
3	3	[	1	2	**<u>3</u>**	4	5	]
4	2	[	**1**	**2**	**3**	**4**	**5**	]

Quick Sort

```
1  public static void QuickSort(Item v[], int esquerda, int direita) {
2    Item temp, pivo;
3    int i, j;
4    i = esquerda;
5    j = direita;
6    pivo = v[(esquerda + direita) / 2];
7
8    while (i <= j) {
9      while (v[i].chave() < pivo.chave() && i < direita) i++;
10     while (v[j].chave() > pivo.chave() && j > esquerda) j--;
11
12     if (i <= j) {
13       temp = v[i];
14       v[i] = v[j];
15       v[j] = temp;
16       i++;
17       j--;
18     }
19   }
20
21   if (j > esquerda)
22     QuickSort(v, esquerda, j);
23
24   if (i < direita)
25     QuickSort(v, i, direita);
26 }
```

Quick Sort

- Número de Comparações
  - Melhor Caso: $n \times \log n$
  - Pior Caso: n^2
- Número Movimentações
  - Melhor Caso: $n \times \log n$
  - Pior Caso: $n \times \log n$

Quick Sort

- Vantagens:
  - É extremamente eficiente para ordenar arquivos de dados.
  - Necessita de apenas uma pequena pilha como memória auxiliar.
  - Requer cerca de $n \times \log n$ comparações em média para ordenar n itens.
- Desvantagens:
  - Tem um pior caso n^2 comparações.
  - Sua implementação é muito delicada e difícil: Um pequeno engano pode levar a efeitos inesperados para algumas entradas de dados.
  - O método não é estável.

Exercício

- Dada a sequência de números: [3 4 9 2 5 1 8] Ordene em ordem crescente utilizando o algoritmo *QuickSort*, apresentado a sequência dos números a cada passo.

Referência

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.